

ECE 132A — Digital Communications Systems

Course Project 2026 — Personalized Simulation

Total: 90 points Teams of 1–3 Deliverables: PDF report + .m file + Appendix A CSV

Each team has a unique team token. Your token (e.g., ECE132A-2026-A7Q2-9XF1) was emailed to you on day 1. It seeds *everything* you work with: your modulation, your channel, your noise level, your code, and a binary received-signal file (`rx_team.<token>.mat`) generated by the instructor's reference simulator. Your team's problem is unique. Sharing data files between teams will be detected by automated comparison and treated as an academic-integrity violation.

Contents

1 Overview	2
2 Section 0 — Your Team's Distributed Data	2
3 Phase 1 — Modulation Theory & AWGN Simulation [15 pts]	3
3.1 1a. Pre-Simulation Derivation [5 pts]	3
3.2 1b. Monte Carlo BER Simulation [8 pts]	3
3.3 1c. Discrepancy Analysis [2 pts]	4
4 Phase 2 — Matched Filtering & Eye Diagrams [10 pts]	4
4.1 2a. Build the Matched Filter [5 pts]	4
4.2 2b. Eye Diagrams [5 pts]	4
5 Phase 2.5 — Channel Estimation & Equalization [25 pts]	4
5.1 2.5a. Pre-Implementation Derivation [5 pts]	4
5.2 2.5b. Channel Estimation [8 pts]	5
5.3 2.5c. ZF and MMSE Equalizer Design [8 pts]	5
5.4 2.5d. Bug-Hunting [4 pts]	5
6 Phase 3 — Fading Channels [15 pts]	5
6.1 3a. Written Derivations [5 pts]	5
6.2 3b. AWGN vs. Rayleigh vs. Rician BER [10 pts]	6
7 Phase 5 — End-to-End Recovery [25 pts]	6
7.1 5a. Identify and Compensate Your Anomaly [8 pts]	6
7.2 5b. End-to-End BER on Your Data [8 pts]	6
7.3 5c. Hidden Message Search [4 pts]	7
7.4 5d. Reflection [5 pts]	7
8 Deliverables	7
Useful Formulas	8

Overview

The project is structured around **a recorded received signal you did not generate**. The signal was produced by an instructor-only simulator that applied a unique frequency-selective channel, a unique noise level, and one small unique impairment to your team's transmitted waveform. Your task is to characterize, equalize, and demodulate this signal end-to-end, and to *report specific numerical answers* extracted from your data.

- **Phase 1 [15 pts]** — Modulation theory and AWGN simulation.
- **Phase 2 [10 pts]** — Matched filtering and eye diagrams.
- **Phase 2.5 [25 pts]** — Channel estimation & equalization on your data file.
- **Phase 3 [15 pts]** — Fading channels.
- **Phase 5 [25 pts]** — End-to-end recovery from your data file.

In-person exam tie-in. One question on the in-class final exam (no electronics, closed book) will test your understanding of material covered by this project. Plan to do the work yourself: the exam question is designed so that students who personally completed the analysis can answer it cleanly, while students who outsourced the project to AI tools will not.

Policy on AI assistance. You may use AI tools (LLMs, code assistants) as you would use a textbook or a colleague: to look up a derivation, get unstuck on syntax, or check intuition. You may *not* submit AI-generated content as your own work without verifying every line and every number. The grading mechanics described below (per-team data, reproducibility check, critique task, Appendix A summary table) are designed so that you cannot pass this project without genuine understanding even with full AI access. If unsure whether a use is permitted, ask the instructor before the deadline, not after.

Reproducibility check (binding). Your submitted `.m` file will be re-executed by the grader against your assigned data file. Every numerical answer in your report must agree with the script's output to within $\pm 0.5\%$. Material discrepancies will be treated as fabricated results under the academic-integrity policy. Plan your code accordingly: *the numbers in the report must be the numbers your script produces*.

Section 0 — Your Team's Distributed Data

Each team receives three files (with `<T>` replaced by your team token):

- `rx_team_<T>.mat` — complex baseband received samples, oversampled at `samples_per_symbol = 4`, plus a 256-symbol BPSK training preamble at the start (also stored separately in this file as `preamble_symbols`).
- `params_team_<T>.json` — your modulation, target BER, operating $\mathcal{E}_b/N_0$, and pulse-shape parameters. Read this; do not modify.
- `truth_team_<T>.txt` — the transmitted bits (the bit sequence that was modulated and transmitted), in plain text, one bit per line. Use *only* for computing your final BER — not during receiver design.

The instructor manifest containing the true channel taps, noise variance, and anomaly type is *not* distributed to students.

The first thing you should do in MATLAB is:

Listing 1: Loading your team's data

```
1 load(['rx_team_' team_token '.mat']);
2 params = jsondecode(fileread(['params_team_' team_token '.json']));
3 truth_bits = load(['truth_team_' team_token '.txt']); % info bits
```

The variables you can rely on inside the `.mat`:

Variable	Type	Description
<code>rx_signal</code>	complex 1×N	received samples (your input)
<code>samples_per_symbol</code>	double	oversampling factor (= 4)
<code>preamble_symbols</code>	complex 1×256	known training symbols (± 1)
<code>rrc_rolloff</code>	double	RRC roll-off ($\alpha = 0.25$)
<code>rrc_span_symbols</code>	double	RRC filter span (= 10 symbols)
<code>modulation</code>	char	e.g., '16QAM'
<code>payload_length_bits</code>	double	transmitted bit count
<code>target_BER</code>	double	your target BER
<code>EbN0_target_dB</code>	double	nominal E_b/N_0 at which the data was generated

What you do not know. The number and values of the channel taps, the exact noise variance applied, the type and value of the small impairment (timing offset / carrier-frequency offset / DC bias / none), and whether your payload contains a hidden ASCII message. **You will infer these from the signal.** The grader knows them all.

Phase 1 — Modulation Theory & AWGN Simulation [15 pts]

1a. Pre-Simulation Derivation [5 pts]

Hand-derived the expressions *before* running any simulation.

For **your team's modulation** (one of BPSK, QPSK, 8-PSK, 16-QAM, 64-QAM — check `params.modulation`):

- Derive the symbol-error probability P_s in AWGN as a function of E_b/N_0 . Show the Q-function or $\text{erfc}(\cdot)$ form.
- Convert to bit-error probability P_b assuming Gray coding. State and justify any approximations (nearest-neighbor union bound, etc.).
- State the asymptotic high-SNR slope of $\log P_b$ vs. E_b/N_0 in dB per decade.

1b. Monte Carlo BER Simulation [8 pts]

Simulate BER vs. E_b/N_0 over $[0, 15]$ dB for your team's modulation. Use at least 10^6 bits per SNR point or until you observe ≥ 100 bit errors, whichever produces more bits. Use Gray coding.

Plot simulated and theoretical curves on a single **semilogy** figure with proper labels, units, and legend.

1c. Discrepancy Analysis [2 pts]

For your modulation, identify the $\mathcal{E}_b/N_0$ range (in dB) where the Monte Carlo estimate is within ± 0.5 dB of theory. Explain what limits the agreement at the low and high ends. Quote a 95% confidence-interval calculation for one SNR point of your choice.

Phase 2 — Matched Filtering & Eye Diagrams [10 pts]**2a. Build the Matched Filter [5 pts]**

Implement an RRC matched filter with parameters from `params.rrc`. Verify (numerically) that the cascade RRC * RRC is approximately Nyquist: the maximum interference from neighboring symbols, sampled at the optimal instant, should be below -30 dB relative to the main lobe. Plot the cascade impulse response and label its zero crossings at integer symbol times.

2b. Eye Diagrams [5 pts]

Using *your* `rx_signal`, generate three eye diagrams of the matched-filter output:

- (i) correct sampling instant;
- (ii) sampling 20% of T_s *early*;
- (iii) sampling 20% of T_s *late*.

Quantify the eye opening (peak-to-peak vertical opening at the optimal sample instant, in dB relative to the symbol amplitude) for each case. Comment on whether the eye is actually closed in any of the three.

Note: the eye opening you measure on your data will reflect both the RRC pulse and your team's frequency-selective channel. Don't expect a textbook diamond shape — and report what you actually see.

Phase 2.5 — Channel Estimation & Equalization [25 pts]

This phase operates on *your* `rx_signal`. You will identify the channel from the training preamble, design two equalizers, and compare their performance on the symbols you can recover.

2.5a. Pre-Implementation Derivation [5 pts]

Complete before implementing.

- (i) Set up the least-squares channel estimation problem: given the symbol-rate matched-filter output samples $y[n]$, $n = 0, \dots, N_p - 1$ (where $N_p = 256$ is the preamble length), the known preamble symbols $p[n]$, and an assumed channel length L_{est} , write the matrix equation $A\hat{h} = b$ for the LS estimate. State the dimensions of A and b , and explain why you discard the first $L_{\text{est}} - 1$ rows.
- (ii) Derive the closed-form MMSE equalizer for an FIR filter of length N targeting decision delay d , given a channel estimate $\hat{h}$ and a symbol-rate noise variance σ_n^2 . Express your answer in matrix form: $w_{\text{MMSE}} = \dots$. Show the regularization term explicitly.

2.5b. Channel Estimation [8 pts]

Implement the LS estimator from 2.5a. Use $L_{\text{est}} = 8$ (deliberately larger than the true channel length, which is between 3 and 7 taps).

Required numerical answers (will appear in Appendix A):

- Estimated channel taps $\hat{h}[0], \dots, \hat{h}[L_{\text{est}} - 1]$, each reported as a complex number to four decimal places.
- Frequency of the deepest spectral null in the magnitude response of $\hat{H}(e^{j2\pi f})$, normalized to f_s , to three decimal places. (Sweep f on a fine grid; report the global minimum location.)
- Depth of that null in dB.
- LS residual norm $\|A\hat{h} - b\|_2$.

Discussion: based on your estimated taps, do you see any taps that are statistically zero (within the noise floor estimated from the LS residual)? If so, what does that imply about the true channel length, and does it match your expectation?

2.5c. ZF and MMSE Equalizer Design [8 pts]

Design two FIR equalizers of length $N_{\text{eq}} = 21$:

- Zero-forcing: w_{ZF} minimizing $\|Hw - e_d\|_2^2$ (no regularization).
- MMSE: w_{MMSE} from 2.5a, with σ_n^2 estimated from your data (see hint below).

Hint for σ_n^2 : after channel estimation, the residual $\mathbf{r} = A\hat{h} - b$ is dominated by noise in the regime where L_{est} overestimates the true L . Estimate $\sigma_n^2 \approx \|\mathbf{r}\|_2^2 / (\text{rows}(A) - L_{\text{est}})$. Justify any other estimator you use.

Required numerical answers:

- ZF equalizer-output SNR at the slicer input, in dB to one decimal place.
- MMSE equalizer-output SNR at the slicer input, in dB to one decimal place.
- Difference (MMSE – ZF) in dB.

Discussion: explain in terms of *your specific channel taps* (in particular, your null frequency and depth) why MMSE outperforms ZF (or, in unusual cases, doesn't). Quote the gap quantitatively.

2.5d. Bug-Hunting [4 pts]

The starter code includes a function `mmse_equalizer_starter` that contains **one intentional sign error** affecting the regularization term. Identify the bug, explain its consequences (does it produce ZF behavior? does it amplify or suppress noise? at what SNR is the effect most damaging?), and submit a corrected `mmse_equalizer_fixed` function. Show the slicer-input SNR for the buggy and fixed versions on your data file, in dB.

Phase 3 — Fading Channels [15 pts]**3a. Written Derivations [5 pts]**

Complete before simulating.

- (i) Derive the Rayleigh-envelope PDF starting from a complex Gaussian $h = h_I + j h_Q$ with $h_I, h_Q \sim \mathcal{N}(0, \sigma^2)$ i.i.d. Show the Jacobian.
- (ii) Define the Rician K -factor in terms of the LOS-to-diffuse power ratio. Sketch the envelope PDF for $K = 0$ and $K = 10$ on the same axes.

3b. AWGN vs. Rayleigh vs. Rician BER [10 pts]

Pick a Rician K -factor for this phase as $K = (\text{last digit of your team token, hex}) \bmod 8 + 1$. State your K on the report cover.

Simulate BER vs. ϵ_b/N_0 over $[0, 30]$ dB (step 2 dB) for BPSK over AWGN, Rayleigh flat, and Rician flat fading with your K . Overlay the closed-form Rayleigh BER:

$$P_b^{\text{Rayleigh}} = \frac{1}{2} \left(1 - \sqrt{\bar{\gamma}/(1 + \bar{\gamma})} \right), \quad \bar{\gamma} = \epsilon_b/N_0. \quad (1)$$

Required numerical answers:

- Rayleigh fading penalty at $\text{BER} = 10^{-3}$, in dB.
- ϵ_b/N_0 at which Rician with your K matches AWGN to within 1 dB, in dB to 0.5 dB precision.

Phase 5 — End-to-End Recovery [25 pts]

This is the integration phase. Use your full receiver chain (matched filter, channel estimation, MMSE equalizer, demodulation) on your `rx_signal`, and compare to the truth bits.

5a. Identify and Compensate Your Anomaly [8 pts]

Your data file contains exactly *one* of: a small carrier-frequency offset (CFO), a small DC bias, a fractional-symbol timing offset, or no anomaly at all. Your task:

- (i) Devise a method to identify which of the four cases applies to your data. *Hint:* after pulling out the preamble samples, look at their amplitude statistics and time-evolution.
- (ii) Estimate the value of the anomaly (offset value, bias magnitude, or timing fraction) and compensate for it.
- (iii) Show, with one figure, evidence of the anomaly before correction and evidence of its absence after correction.

Required numerical answer: the inferred anomaly type and value, to the precision your detector supports.

5b. End-to-End BER on Your Data [8 pts]

Process your full `rx_signal` through:

matched filter $\rightarrow$ anomaly comp $\rightarrow$ equalize $\rightarrow$ demodulate $\rightarrow$ recover transmitted bits.

Compare to `truth.bits` and report:

- Measured BER on the demodulated bits.

- Whether your measured BER agrees with theory at your team's `EbN0_target_dB`, and a quantitative explanation of any gap (e.g., residual ISI from the equalizer, imperfect anomaly compensation, channel-estimation error).

5c. Hidden Message Search [4 pts]

Half the team data files contain a short ASCII string embedded in the transmitted payload at byte offset 64 (i.e., at transmitted bits $[512, 512 + 8 \cdot |\text{msg}|]$). After demodulation, scan that range of your recovered bits, group into bytes (left-MSB), and check for a printable string of length ≥ 8 . If your team's data contains a hidden message, recover and report it. If not, report *none* and explain how you verified absence (e.g., the bit error rate in that range matches the rest of the payload).

5d. Reflection [5 pts]

In 1–2 paragraphs (no more): which step in your pipeline contributed the most to your final BER? If you had one more week, what would you change first, and what improvement (in dB or in BER) would you expect, quantitatively, on your data?

Deliverables

Submit in **one zip file** named after your team (one file per team) `team_<Token>.zip`:

- ☐ `CommProject_Report.pdf` — the report described below.
- ☐ `CommProject.m` — a single MATLAB script that, when run from a folder containing your data files, produces all figures and prints all numerical answers in your report.
- ☐ `AppendixA.csv` — the numerical-answer summary table (see below) in two-column form (`field`, `value`).

Report Structure

- ☐ Cover page: team token, member names, declared K from Phase 3b.
- ☐ Phase 1: derivations, BER plot, discrepancy analysis.
- ☐ Phase 2: matched-filter verification, eye diagrams (3), eye-opening measurements.
- ☐ Phase 2.5: derivations, estimated taps, spectral-null location, ZF vs. MMSE comparison, bug analysis.
- ☐ Phase 3: derivations, fading BER plot, fading-penalty and Rician-asymptote answers.
- ☐ Phase 5: anomaly identification + correction figure, end-to-end BER, hidden-message recovery (or absence), reflection.
- ☐ **Appendix A: Numerical Answer Summary Table.** See template below. Every row corresponds to one boxed answer earlier in the report. The grader will diff this table against the output of your `.m` file.

Appendix A Template

Reproduce this table; one row per measurement. The grader's verification script reads the CSV.

field	value
team_token	ECE132A-2026-...
modulation	...
target_BER	...
EbN0_target_dB	...
channel_taps_estimated	$\hat{h}[0], \hat{h}[1], \dots, \hat{h}[L_{\text{est}} - 1]$ as complex
null_freq	f_0 (normalized to f_s , three decimals)
null_depth_dB	dB
sigma2_estimated	noise variance estimate
ZF_SNR_dB	dB to 0.1
MMSE_SNR_dB	dB to 0.1
MMSE_minus_ZF_dB	dB to 0.1
mmse_bug_explanation	one short sentence
fading_penalty_dB	dB at BER = 10^{-3}
Rician_K_used	integer 1–8
anomaly_type	none / CFO / DC_bias / timing_offset
anomaly_value	value or 0
measured_BER	scientific notation
hidden_message	ASCII string or “none”

Useful Formulas

$$Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty e^{-t^2/2} dt \quad (2)$$

$$\text{BPSK / QPSK BER} = Q\left(\sqrt{2\mathcal{E}_b/N_0}\right) \quad (3)$$

$$\text{16-QAM BER (approx.)} \approx \frac{3}{4} Q\left(\sqrt{\frac{4}{5}\mathcal{E}_b/N_0}\right) \quad (4)$$

$$\text{LS channel estimate} \quad \hat{h} = (A^H A)^{-1} A^H b \quad (5)$$

$$\text{MMSE equalizer} \quad w = (H^H H + \sigma_n^2 I)^{-1} H^H e_d \quad (6)$$

$$\text{Rayleigh BER} = \frac{1}{2} \left(1 - \sqrt{\bar{\gamma}/(1 + \bar{\gamma})}\right) \quad (7)$$

MATLAB quick references: `qfunc(x)`, `jsondecode(fileread(...))`, `conv(a,b)`, `filter(b,a,x)`.